

boaz_학기 세션 과제(2)

LoRA

1. Abstract
2. Introduction
 - 2.1. 기존 방식 한계
 - 2.2. LoRA 목적
 - 2.3. Problem Statement
 - 2.4. **Aren't Existing Solutions Good Enough?**
3. Methods
 - 3.1. Low-Rank-Parametrized Update Matrics
 - 3.2. **Applying LoRA to Transformer**
 - 3.3. Practical Benefits of LoRA
4. Experiments
 - 4.1. 실험 설계
 - 4.2. Result
5. Conclusion
 - 5.1. **Understanding the Low-Rank Updates**

RAG

1. Abstract
2. Introduction
 - 2.1. 등장 배경
 - 2.2. RAG 제안
3. Methods
 - 3.1. Models
 - 2.2. Retriever (DPR)
 - 2.3. Generator(BART)
 - 2.4. Training
 - 2.5. Decoding
4. Experiments
 - 4.1. Dataset
 - 4.2. **Open-domain Question Answering**
 - 4.3. **Abstractive Question Answering**
 - 4.3. **Jeopardy Question Generation**
 - 4.4. Fact Verification
 - 4.5. Additional Results
5. Conclusion

LoRA

1. Abstract

- 일반적으로 사전 학습된 모델의 모든 파라미터를 업데이트하는 full fine-tuning으로 수행
- 모델 파라미터가 커짐에 따라 전체 파라미터를 미세 조정하는 데 막대한 비용 발생
- 가중치 업데이트 과정이 Low intrinsic dimension을 가진다는 가설 하에 학습 가능한 작은 행렬만을 추가해 연산 효율을 극대화

2. Introduction

2.1. 기존 방식 한계

- 일부 파라미터만 조정하거나 새로운 task를 위한 외부 모듈을 학습해 완화하려는 시도
- 모델 깊이를 확장하거나 모델의 사용 가능한 sequence 길이의 제한과 추론 지연 (latent latency) 발생
- 이러한 방법은 fine-tuning baseline과 일치하지 않는 경우가 많아 효율성과 모델 품질 간 trade-off 발생

2.2. LoRA 목적

- 사전 학습된 가중치를 고정된(frozen) 상태로 유지하되 업데이트가 필요한 부분만 저차원 행렬로 분해하여 학습함
- 파라미터 학습 과정에서 전체 가중치 행렬을 업데이트하지 않고 2개의 저차원 랭크로 대체해 행렬을 업데이트

⇒ 학습 가능한 파라미터 수를 줄여 메모리 및 계산 비용을 줄이면서도 모델의 성능 유지가 가능

⇒ LLM의 pre-trained 파라미터를 downstream task에 더 효율적으로 adaptation 하는 방식

2.3. Problem Statement

$$\Phi_0 \rightarrow \Phi_0 + \Delta\Phi$$

- Φ 로 parameterize된 사전 학습된 autoregressive model $P_{\Phi}(y|x)$ 가 주어졌다고 가정
 - $P_{\Phi}(y|x) \rightarrow$ transformer 아키텍처를 기반으로 하는 multi-task learner(GPT)
- 기존 방식 : $P_{\Phi}(y|x)$ 을 downstream task에 맞게 모든 파라미터를 업데이트
 - 사전 학습된 모델이 큰 경우(GPT-3(175B)) fine-tuning된 모델의 많은 인스턴스를 저장, 배포하는것은 어려움
- LoRA 제안 : 각 task별로 파라미터 Φ 로 인코딩되는 효율적 접근 방식 제안
 - low-rank 표현을 사용하여 계산 및 메모리 효율적인 $\Delta\Phi$ 를 인코딩해 학습 가능한 파라미터 수가 기존보다 0.01% 정도로 작을 수 있음

2.4. Aren't Existing Solutions Good Enough?

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4 $\pm$ 0.8	338.0 $\pm$ 0.6	19.8 $\pm$ 2.7
Adapter ^L	1482.0 $\pm$ 1.0 (+2.2%)	354.8 $\pm$ 0.5 (+5.0%)	23.9 $\pm$ 2.1 (+20.7%)
Adapter ^H	1492.2 $\pm$ 1.0 (+3.0%)	366.3 $\pm$ 0.5 (+8.4%)	25.8 $\pm$ 2.2 (+30.3%)

- 모델 adaptation 전략
 - Adapter layer 추가 - GPU 병렬 처리 불가능
 - Prompt/prefix tuning - 최적화가 어렵고 사용가능한 시퀀스 길이가 줄어들어 프롬프트 튜닝 성능이 감소함

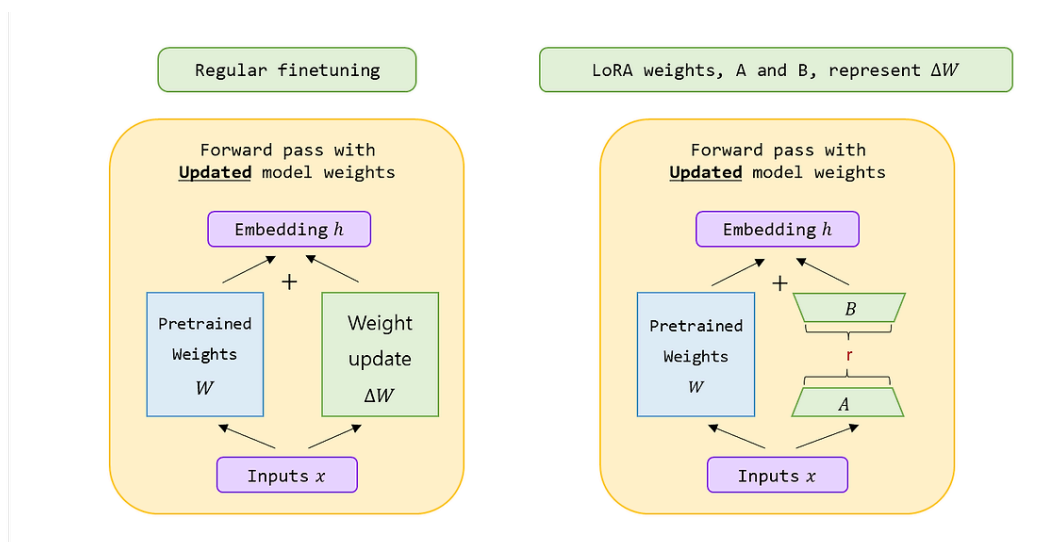
$\Rightarrow$ 모두 대규모 모델이나 latency에 민감한 시나리오에서 한계가 존재

3. Methods

3.1. Low-Rank-Parametrized Update Matrices

$$h = W_0x + \Delta Wx = W_0x + BAx$$

- h (Output)
- W_0 : pretrained weight : : 사전학습 가중치 행렬, freeze
- ΔW : 추가하고 싶은 변화량
- ΔW 를 직접 계산하는것이 아닌 2가지 행렬 B, A 를 곱해 그 역할을 대신함
 - 오직 A, B 만 학습시킴
- 행렬 곱셈을 수행하는 레이어가 포함된 신경망(full-rank)
- 특정 task에 adapt할때 pretrained 모델은 low intrinsic dimension(내재적차원)을 가짐
 - LoRA는 가중치 업데이트도 저차원 decomposition으로 표현



- Pretrained Weight W
- A, B : 저차원 행렬
 - input x 가 들어오면 W 와 A, B 로 보냄
 - W 는 고정된 채로 ΔW 를 근사하는 저차원 행렬 A, B 를 학습
 - $\Delta W = B \cdot A$ 값이 W 에 더해져 W 가 간접적으로 업데이트됨

- 행렬을 A,B 2개로 나누어 연산량을 획기적으로 줄임
- pretrained 모델의 전체 가중치 행렬을 업데이트 하지 않고 더 적은 수의 중요한 파라미터만 사용한 rank decomposition matrices 학습
 - pretrained 모델을 downstream task에 더 효율적으로 튜닝

3.2. Applying LoRA to Transformer

- 학습 가능한 파라미터 수를 줄이기 위해 신경망에서 가중치 행렬의 부분집합에 LoRA 적용
 - Transformer 아키텍처(self-attention 모듈)
 - W_q (Query)
 - W_k (Key)
 - W_v (Value)
 - W_o (Output)
 - 단순성과 파라미터 효율성을 위해 downstream task에 대한 attention 가중치만 튜닝하기 (W_q, W_v)
- ⇒ **W_q (Query) + W_v (Value)만 LoRA 적용, MLP 모듈은 고정**

3.3. Practical Benefits of LoRA

- 메모리 사용량 - GPT-3 175B의 경우 VRAM 소비를 1.2TB → 350GB로 감소
- 체크포인트 크기 - 350GB → 35MB로 감소
- 학습 속도 - 파라미터에 대한 기울기 계산이 필요없어 전체 fine-tuning에 비해 25% 학습 속도 향상
- 추가적인 inference latency 발생 X

4. Experiments

4.1. 실험 설계

- Task
 - NLU - 모델이 문장 의미를 이해하는가?
 - NLG - 모델이 새로운 텍스트를 잘 생성하는가?
- 평가기준
 - GLUE benchmark - 감정 분석/문장 유사도/자연어 추론 평가 기준

4.2. Result

- 다양한 adaptation 방법을 적용한 RoBERTabase, RoBERTalarge, DeBERTaXXL 에 대한 GLUE 벤치마크

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1$\pm$2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3$\pm$3	90.8 $\pm$ 1	86.6$\pm$7	91.5$\pm$2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$3	91.6 $\pm$ 1	87.4$\pm$2.5	92.6$\pm$2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3$\pm$1.0	94.8$\pm$2	91.9$\pm$1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3	96.6$\pm$2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$3	91.6 $\pm$ 2	85.2$\pm$1.1	92.3$\pm$5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2	96.9 $\pm$ 2	92.6$\pm$6	72.4$\pm$1.1	96.0$\pm$1	92.9$\pm$1	94.9$\pm$4	93.0$\pm$2	91.3

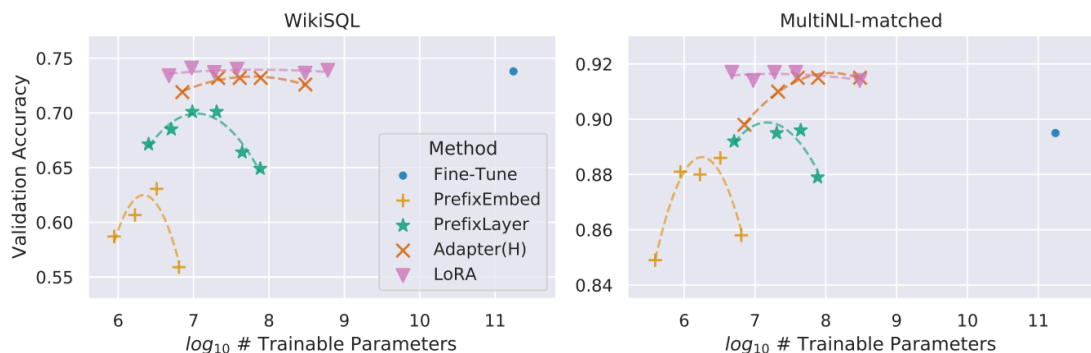
- E2E NLG Challenge에서 다양한 adaptation 방법을 적용한 GPT-2 medium (M)과 GPT-2 large (L)에 대한 비교 결과

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

- 다양한 adaptation 방법을 적용한 GPT-3 175B의 성능 비교 결과

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

- 다양한 adaptation 방법을 적용한 GPT-3 175B의 정확도와 파라미터 개수에 대한 그래프



5. Conclusion

5.1. Understanding the Low-Rank Updates

- Transformer의 어떤 가중치 행렬에 LoRA를 적용해야 하는가?

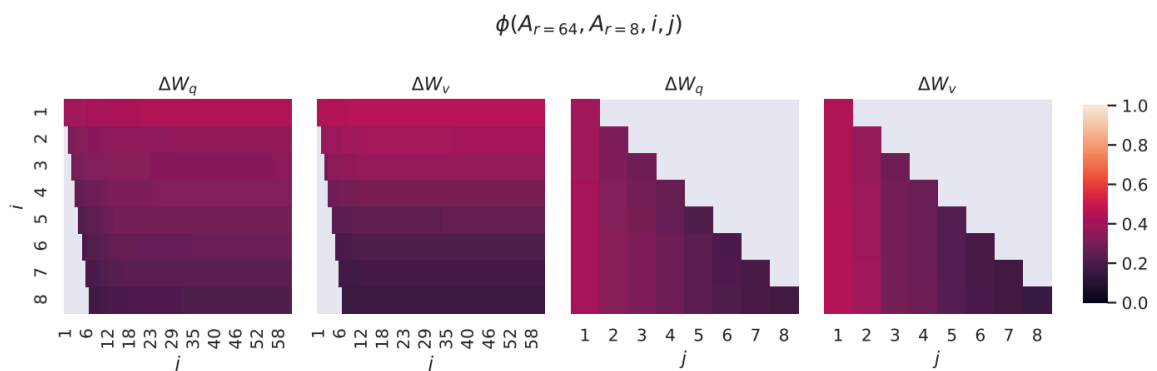
	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

- 학습 파라미터 - GPT-3 175B 중 18M 파라미터만 사용(0.01%)

- 실제로 low-rank 구조를 가지는가?

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

- 다양한 r 에 대한 정확도
- 작은 r 에서도 좋은 성능
 - $r=1, r=2$ 일때 고rank와 큰 차이 없음



- $r = 64, r = 8$ 일때 ΔW (학습방향)비슷 → 짧은 요약본도 긴 본문의 내용을 담고 있음
- $r = 1$ 만 학습시켜도 유사도가 0.5가 넘어 모델의 학습 방향을 어느정도 잘 잡을 수 있음

- 작은 r 값이 모든 task에 작동하는 것은 아님(pre-training 언어와 fine-tuning 언어가 다를 경우)

- **Adaptation 행렬 ΔW 는 W 와 어떻게 비교되는가**

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

- ΔW 는 랜덤 행렬에 비해 W 와 더 강한 상관관계를 가지며, 이는 ΔW 가 이미 W 에 있는 일부 feature를 증폭함을 나타냄
- 기존 모델방향과 다른 특징을 매우 크게 강화시킴

RAG

1. Abstract

- LLM이 pretrained 지식에만 의존하지 않고 외부 데이터베이스, 문서에서 관련 정보를 실시간으로 검색해 이를 기반으로 응답을 생성하는 기술
- Retrieval, Augmentation, Generation을 기반으로 정확하고 구체적인 답변 생성
- 할루시네이션 현상 감소, 실시간 지식 업데이트 가능
- Natural Questions, WebQuestions, CuratedTrec에서 SOTA 달성

2. Introduction

2.1. 등장 배경

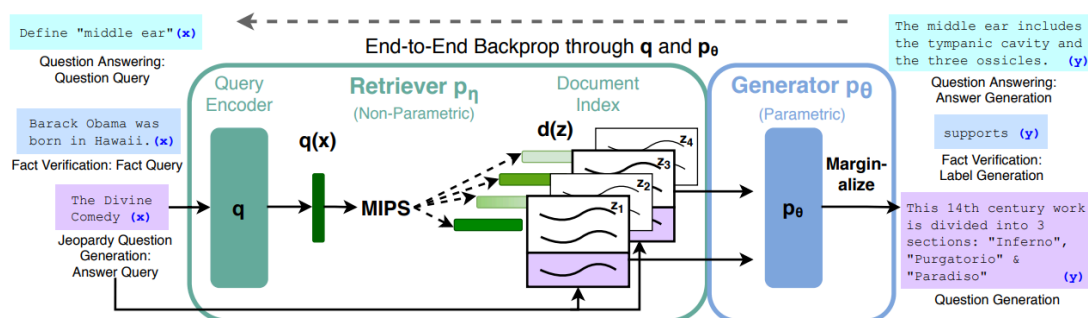
- 기존 모델(GPT, BERT, BART..)

- 사전 학습된 신경 언어 모델 (외부 문서나 지식베이스를 실시간으로 검색하지 않아도 학습된 파라미터 속에 간접적으로 저장된 지식을 기반으로 답변을 생성하는 모델)
- 메모리 확장, 수정이 어려움
- 예측 결과에 대한 명확한 근거 제공 어려움
- hallucination 현상 발생
- Parametric memory, non-parametric으로 일부 해결
 - 모델의 파라미터 안에 저장된 지식(parametric)과 외부 문서를 검색해 활용하는 지식(non-parametric)을 결합한 하이브리드 모델
 - 지식 수정, 확장 가능
 - 언어모델이 가진 문장 생성, 추론 능력 활용 불가

2.2. RAG 제안

- 범용 fine-tuning방법을 제안해 사전 학습 모델에 non-parametric memory부여
 - parametric memory - seq2seq transformer
 - non-parametric memory - 위키피디아의 dense한 벡터 인덱스
- parametric memory와 non-parametric memory가 모두 사전 학습되기에추가 학습 없이도 액세스 메커니즘을 사용함으로써 지식에 접근

3. Methods



3.1. Models

- RAG는 입력 시퀀스 x 를 사용해 텍스트 문서 z 를 검색하고 이를 활용해 target 시퀀스 y 생성
 1. **Retriever** $p_{\eta}(z|x)$: 파라미터가 η 이며 주어진 쿼리 x 에 대하여 텍스트 구절에 대한 분포를 반환
 2. **Generator** $p_{\theta}(y_i|x, z, y_{1:i-1})$: 이전 토큰들 $y_{1:i-1}$, 원래 입력 x , 검색된 구절 z 의 컨텍스트를 기반으로 현재 토큰 y_i 생성
- top-K 근사 + 생성
 - RAG-Sequence : 동일한 하나의 문서를 사용해 전체 토큰 예측

$$\begin{aligned}
 p_{\text{RAG-Sequence}}(y|x) &\approx \sum_{z \in \text{top-k}(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) \\
 &= \sum_{z \in \text{top-k}(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})
 \end{aligned}$$

⇒ 검색기로 top-K 문서를 검색, 하나의 latent variable로 취급해 top-K 근사를 통해 $p(y|x)$ 를 얻음

⇒ 상위 K개 문서는 retriever로 검색되고 generator은 각 문서에 대한 출력 시퀀스 확률을 생성

- RAG-Token : 다른 문서를 기반으로 각 대상 토큰 예측, RAG-Sequence 모델 과정과 같지만 토큰마다 이 과정을 반복

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-k}(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

⇒ 각 토큰에 대해 다른 latent 문서를 사용하고 generator은 답변을 생성할때 여러 문서에서 콘텐츠 선택

⇒ 상위 K개 문서는 retriever로 검색되고 generator은 각 문서에 대한 출력 토큰에 대한 분포 생성 후 이 프로세스를 다음 출력 토큰에 대해 반복

- End-to-end 학습으로 함께 학습 및 파인튜닝 가능

2.2. Retriever (DPR)

- 사전 학습된 신경망 기반 검색기

$$p_{\eta}(z|x) \propto \exp(\mathbf{d}(z)^{\top} \mathbf{q}(x))$$

- bi-encoder 아키텍처를 따름
 - $\mathbf{d}(z)$: 문서 인코더로 생성된 dense 표현
 - $\mathbf{q}(x)$: 쿼리 인코더로 생성된 query 표현
 - 검색 확률 : x 에 대해 문서 z 가 정답 생성에 도움 될 가능성을 확률적으로 나타냄
 - MIPS 문제 : 각 문서 z 를 벡터로 표현 후 가장 큰 값을 가지는 top-K문서를 찾는 것

2.3. Generator(BART)

- 사전학습된 모델
- $p_{\theta}(y_i|x, z, y_{1:i-1})$
 - 임의의 인코더-디코더를 사용해 모델링
 - 사전 학습된 seq2seq transformer인 BART-large 사용
 - BART에서 생성할때 입력 x 와 문서 z 는 간단히 결합됨
 - BART generator 파라미터 $\theta \rightarrow$ parametric memory

2.4. Training

- 학습 방향 : 정답에 대한 negative marginal log-likelihood 최소화
 - 어떤 문서를 검색하는지 supervision없이 retriever, generator 공동학습
- 최적화 : Adam을 사용한 SGD 사용
- 파인튜닝 :
 - 문서 인코더를 주기적으로 업데이트하는 것은 비용이 많이 듦
 - 문서 인코더, 인덱스는 고정하고 쿼리 인코더, generator만 파인튜닝

2.5. Decoding

- 테스트 시 RAG-Sequence와 RAG-Token이 다른 방법으로 $\operatorname{argmax}_p(y|x)$ 근
- RAG-Token

$$p'_\theta(y_i|x, y_{1:i-1}) = \sum_{z \in \text{top-k}(p(\cdot|x))} p_\eta(z_i|x) p_\theta(y_i|x, z_i, y_{1:i-1})$$

- Beam search : 토큰을 하나씩 생성하며 가능성 높은 후보들을 여러 개 유지하며 문장을 만들어가는 디코딩 방식
- 디코딩하려면 $p'_\theta(y_i|x, y_{1:i-1})$ 을 beam search 디코더에 넣으면 됨
- RAG-Sequence
 - $p(y|x)$ 는 토큰별 확률이 아닌 전체 문장에 대한 확률만 존재
 - 문장 전체가 있어야하므로 한번의 beam search로 해결 못함
 - Thorough Decoding
 1. 각 문서 z 에 대해 beam search 실행하여 각 가설 채점
 2. 가설들의 집합 Y 생성
 3. beam에 y 가 나타나지 않은 문서 z 에 대해 각각의 확률을 곱하고 평균냄
 4. 가장 확률 높은 문장 y 선택
 - Fast Decoding
 1. beam search 중 생성되지 않은 y 에 추가 근사 실행
 2. 가장 확률 높은 문장 y 선택

4. Experiments

4.1. Dataset

- 문서 데이터 : 2018년 12월 위키피디아 덤프
- 임베딩 : FAISS (facebook vector 라이브러리)
- 문서 검색 방식 : MIPS (Top-K 문서 검색)

4.2. Open-domain Question Answering

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

- Open-Domain QA에 대한 테스트 점수를 비교한 표
 - NQ - 구글 검색 데이터, 일상정보 검색 데이터
 - TQA - 퀴즈, 웹사이트, 복잡한 질문
 - WQ - 구글 검색 엔진에 입력한 질문, 간단하고 명료
 - CT - 정보 검색 및 질의응답 성능 평가
- 광범위한 주제에 걸친 질문에 대한 대답
 1. Extractive QA : 문서에서 정답 span 추출(Generation X)
 2. Closed-Book QA : 모델 파라미터에 저장된 지식만 사용(Retrieval X)
- RAG SOTA 달성

4.3. Abstractive Question Answering

- NLG 평가 - 자연어 생성 능력(질문에 대해 자유형식의 문장 생성)
- dataset - MSMARCO NLG dataset task v2.1 : 검색 기반 QA dataset
- 정성적 평가시 RAG generation이 자연스러움 (BART-hallucination 발생)
- RAG가 BART보다 더 다양한 문장 생성

4.3. Jeopardy Question Generation

- 정답이 주어지고 해당 정답에 대한 사실 기반 문장으로 질문 생성
- 평가
 - Q-BLUE-1 : 엔티티가 얼마나 잘 포함됐는지

- 인간 평가 : 사실성(신뢰가능한 외부자료가 근거인지) 구체성(입력과 출력 사이 상호의존도가 높은지)

4.4. Fact Verification

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

- FEVER : Retrieval + 어려운 추론 문제
- 어떤 문서가 근거인지 알려주지 않음(supervision X)
 - Supported(주장 지지)
 - Refused(주장 반박)
 - Not Enough info(근거 부족)
- 검색된 문서에 대해 supervision 없이 학습하며 높은 탐색 성능 보임

4.5. Additional Results

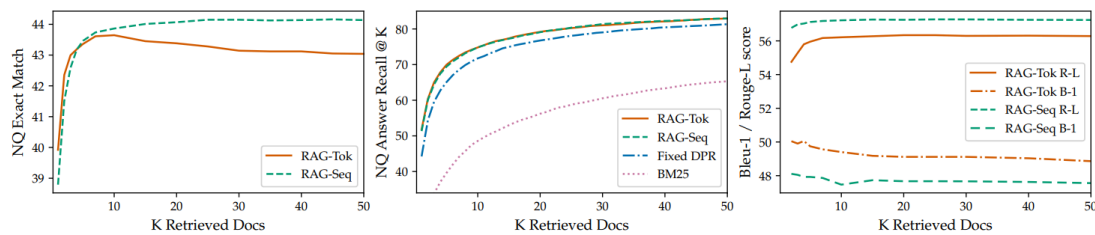
- Generation Diversity

	MSMARCO	Jeopardy QGen
Gold	89.6%	90.0%
BART	70.7%	32.4%
RAG-Token	77.8%	46.8%
RAG-Seq.	83.5%	53.8%

- 전체 tri-gram에 대한 고유 tri-gram의 비율 표현
- RAG-Sequence 생성이 RAG-Token생성보다 더 다양함
- BART보다 훨씬 다양하다
- Retrieval Ablations

Model	NQ	TQA Exact Match	WQ	CT	Jeopardy-QGen B-1 QB-1		MSMarco R-L B-1		FVR-3 Label Accuracy	FVR-2 Accuracy
RAG-Token-BM25	29.7	41.5	32.1	33.1	17.5	22.3	55.5	48.4	75.1	91.6
RAG-Sequence-BM25	31.8	44.1	36.6	33.8	11.1	19.5	56.5	46.9		
RAG-Token-Frozen	37.8	50.1	37.1	51.1	16.7	21.7	55.9	49.4	72.9	89.4
RAG-Sequence-Frozen	41.2	52.1	41.8	52.6	11.8	19.6	56.7	47.3		
RAG-Token	43.5	54.8	46.5	51.9	17.9	22.6	56.2	49.4	74.5	90.6
RAG-Sequence	44.0	55.8	44.9	53.4	15.3	21.5	57.2	47.5		

- Retrieval 방식에 따라 RAG 성능이 어떻게 달라지는지 비
- Test : k Retrived Documents



- Open-domain QA
 - k가 커질수록 RAG-Sequence 성능 향상
 - k = 10에서 RAG-Token 최대, 점차 감소
- Recall
 - k가 커질수록 Recall 향상
- 생성 문장 평가 점수
 - 정답 문장과 의미적으로 겹치는지
 - 같은 단어가 중복되는지

5. Conclusion

- RAG 텍스트 생성은 모델 파라미터에 저장된 지식 + 외부 문서 검색
- 실제 지식 기반을 활용해 hallucination 감소, 더 정확한 답변 생성
- 외부 지식은 편향되거나 부정확하다는 한계 존재